

Reverse engineering

Mobile



Ruxandra F. Olimid

April 30, 2025

Agenda

- Motivation
- Focus on **Android**
 - From source to execution /binaries
 - Structure, binaries
 - Tools
- Resources

Motivation

”The bad news is that dealing with multi-threaded anti-debugging controls, cryptographic white-boxes, stealthy anti-tampering features, and highly complex control flow transformations is not for the faint-hearted. The most effective software protection schemes are proprietary and won't be beaten with standard tweaks and tricks. Defeating them requires tedious manual analysis, coding, frustration and, depending on your personality, sleepless nights and strained relationships.”

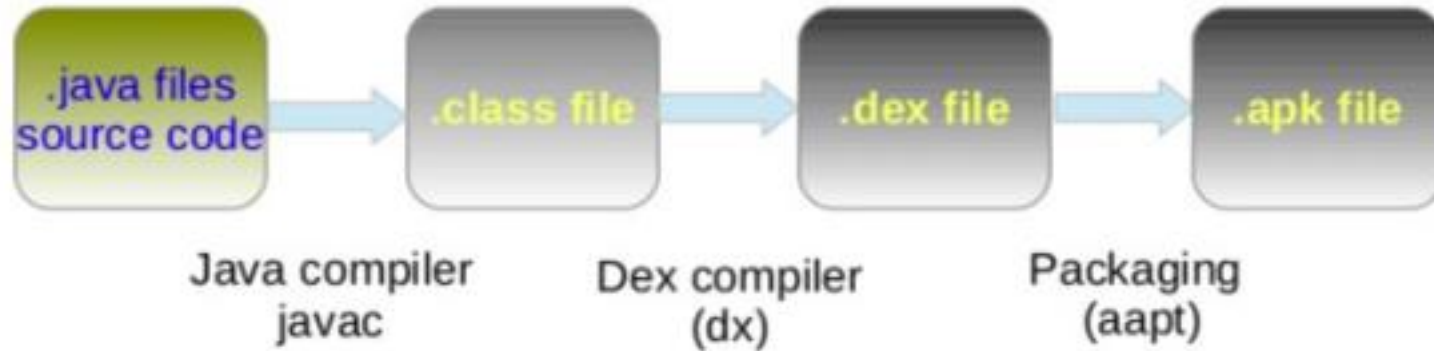
<https://mas.owasp.org/MASTG/0x04c-Tampering-and-Reverse-Engineering>

”Android's openness makes it a favorable environment for reverse engineers, offering big advantages that are not available with iOS .”

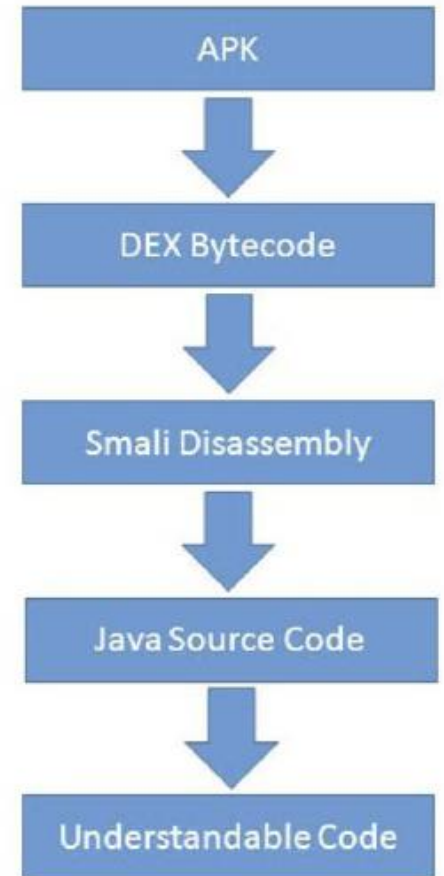
<https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0013/>

From source to executable/binary Android

- https://www.ragingrock.com/AndroidAppRE/app_fundamentals.html



- **smali/baksmali**: <https://github.com/JesusFreke/smali/wiki>
(similar to assemble/disassemble)
- **jadx**: <https://github.com/skylot/jadx>
(dex to java decompiler)



<https://github.com/nowsecure/cybertruckchallenge19/blob/master/slides/CyberTruck19-AndroidSecurity-NowSecure.pdf>
<https://github.com/nowsecure/cybertruckchallenge19/blob/master/slides/CyberTruck21-AndroidSecurity-NowSecure.pdf>

Binaries Android - .dex files

What's New?

Getting Started

Security

Core Topics

Compatibility

Android Devices

Automotive

Reference

Filter

Overview

Dexpreopt and <uses-library> checks

Improvements

Dalvik bytecode format

DEX format

Dalvik executable instruction formats

Constraints

Configuration

ART TI

Implement Signed Config

File layout

Name	Format	Description
header	header_item	the header
string_ids	string_id_item[]	string identifiers list. These are identifiers for all the strings used by this file, either for internal naming (e.g., type descriptors) or as constant objects referred to by code. This list must be sorted by string contents, using UTF-16 code point values (not in a locale-sensitive manner), and it must not contain any duplicate entries.
type_ids	type_id_item[]	type identifiers list. These are identifiers for all types (classes, arrays, or primitive types) referred to by this file, whether defined in the file or not. This list must be sorted by <code>string_id</code> index, and it must not contain any duplicate entries.
proto_ids	proto_id_item[]	method prototype identifiers list. These are identifiers for all prototypes referred to by this file. This list must be sorted in return-type (by <code>type_id</code> index) major order, and then by argument list (lexicographic ordering, individual arguments ordered by <code>type_id</code> index). The list must not contain any duplicate entries.

<https://source.android.com/docs/core/runtime/dex-format>

DALVIK EXECUTABLE

>adb shell dalvikvm -cp /data/hw.zip hw
Hello World!

0 1 2 3 4 5 6 7 8 9 A B C D E F

000: .d .e .x 0A .0 .3 .5 00 6F 53 89 BC 1E 79 B2 4F
010: 1F 9C 00 66 15 23 2D 38 56 65 32 C3 85 81 B4 5A
020: 70 02 00 00 70 00 00 00 78 56 34 12 00 00 00 00
030: 00 00 00 00 DC 01 00 00 0C 00 00 00 70 00 00 00
040: 07 00 00 00 A0 00 00 00 02 00 00 00 0C 00 00 00
050: 01 00 00 00 D4 00 00 00 02 00 00 00 DC 00 00 00
060: 01 00 00 00 EC 00 00 00 64 01 00 00 0C 01 00 00
070: A6 01 00 00 3A 01 00 00 8A 01 00 00 40 01 00 00
080: B4 01 00 00 76 01 00 00 54 01 00 00 6C 01 00 00
090: 57 01 00 00 70 01 00 00 A1 01 00 00 C8 01 00 00
0A0: 01 00 00 00 02 00 00 00 03 00 00 00 04 00 00 00
0B0: 05 00 00 00 06 00 00 00 08 00 00 00 07 00 00 00
0C0: 05 00 00 00 34 01 00 00 07 00 00 00 05 00 00 00
0D0: 2C 01 00 00 04 00 00 00 0A 00 00 00 00 00 01 00
0E0: 09 00 00 00 01 00 00 00 08 00 00 00 00 00 00 00
0F0: 01 00 00 00 02 00 00 00 00 00 00 00 FF FF FF FF
100: 00 00 00 00 D1 01 00 00 00 00 00 00 02 00 01 00
110: 02 00 00 00 00 00 00 00 08 00 00 00 62 00 00 00
120: 1A 01 00 00 6E 20 01 00 10 00 0E 00 01 00 00 00
130: 06 00 00 00 01 00 00 00 03 00 04 .L .h .w .; .j
140: 12 .L .j .a .v .a ./. .l .a .n .g ./. .0 .b .j .e
150: .C .t .; .00 01 .V 00 13 .[.L .j .a .v .a ./. .l
160: .a .n .g ./. .S .t .r .i .n .g .; .00 02 .V .L 00
170: 04 .m .a .i .n 00 12 .L .j .a .v .a ./. .l .a .n
180: .g ./. .S .y .s .t .e .m .; .00 15 .L .j .a .v .a
190: ./. .i .o ./. .P .r .i .n .t .S .t .r .e .a .m .; .
1A0: 00 03 .o .u .t 00 0C .H .e .l .l .0 20 .W .o .r
1B0: .l .d .! 00 12 .L .j .a .v .a ./. .l .a .n .g ./.
1C0: .S .t .r .i .n .g .; .00 07 .p .r .i .n .t .l .n
1D0: 00 00 00 01 00 00 09 8C 02 00 00 00 0C 00 00 00
1E0: 00 00 00 00 01 00 00 00 00 00 00 00 01 00 00 00
1F0: 0C 00 00 00 70 00 00 00 02 00 00 00 07 00 00 00
200: A0 00 00 00 03 00 00 00 02 00 00 00 0C 00 00 00
210: 04 00 00 00 01 00 00 00 D4 00 00 00 05 00 00 00
220: 02 00 00 00 DC 00 00 00 06 00 00 00 01 00 00 00
230: EC 00 00 00 01 20 00 00 01 00 00 00 0C 01 00 00
240: 01 10 00 00 02 00 00 00 2C 01 00 00 02 20 00 00
250: 0C 00 00 00 3A 01 00 00 00 20 00 00 01 00 00 00
260: D1 01 00 00 00 10 00 00 01 00 00 00 DC 01 00 00

HEADER

magic 0x3e357350
adler32 0x82095160
sha1 0x00000000000000000000000000000000
file_size 0x270
header_size 0x70
endian_tag 0x12345678 (little endian)

map_offset 0x10C
size / offsets
strings_ids 0x00 / 0x070
type_ids 0x00 / 0x00C
proto_ids 0x00 / 0x00C
field_ids 0x00 / 0x00C
method_ids 0x00 / 0x00C
class_defs 0x00 / 0x00C
data 0x104 / 0x10C

STRING IDS
(A-Z ORDER)

offset (to string)
0x106 ("Hello World!")
0x10A ("lw;")
0x10B ("Ljava/lang/PrintStream;")
0x10C ("Ljava/lang/Object;")
0x10D ("Ljava/lang/String;")
0x10E ("Ljava/lang/System;")
0x10F ("v")
0x110 ("W")
0x111 ("Ljava/lang/String;")
0x112 ("main")
0x113 ("out")
0x114 ("println")

TYPE IDS
(STRING LIST INDEXES)

1 2 3 4 5 6 8

PROTO IDS

string_id descriptor return type parameters
7 5 0x10C

FIELD IDS

class 0x0 ("lw;")
prototype 0x1 ("Ljava/lang/String;")
name 0x9 ("main")
class 0x1 ("Ljava/lang/PrintStream;")
prototype 0x0 ("Ljava/lang/String;")
name 0x0 ("println")

METHOD IDS

CLASS DEFS

class 0x0 ("lw;")
access_flags 0x1 (PUBLIC)
superclass 0x2 ("Ljava/lang/Object;")
source 0xFFFFFFFF (none)
data_offset 0x102

CODE

registers 2
in_args 1 (words)
out_args 2 (words)
instructions 8 (words)
iget-object v0, "out";
const-string v1, "Hello World!";
invoke-virtual {v0, v1}, Ljava/lang/PrintStream;
return-void

TYPE LIST

size 1
type 6 ("Ljava/lang/String;")
size 1
type 3 ("Ljava/lang/String;")

STRING DATA
(UTF-8)

len / string
00 "lw;"
01 "Ljava/lang/Object;"
02 "Ljava/lang/String;"
03 "W"
04 "main"
05 "Ljava/lang/System;"
06 "Ljava/lang/PrintStream;"
07 "out"
08 "Hello World!"
09 "Ljava/lang/String;"
0A "println"

CLASS DATA

direct methods 1
index 0x0 0x0
flags 0x9 (PUBLIC STATIC)
code offset 0x020C (0x10C, encoded in ulh128)

MAP

count 12
type / size / offset
0x0000 (HEADER) 1 0x0000
0x0001 (STRING) 12 0x0070
0x0002 (TYPE) 7 0x00C
0x0003 (PROTO) 2 0x00C
0x0004 (FIELD) 1 0x00C
0x0005 (METHOD) 2 0x00C
0x0006 (CLASS) 1 0x00C
0x0007 (CODE) 1 0x10C
0x0008 (TYPE LIST) 2 0x10C
0x0009 (STRING DATA) 12 0x10A
0x000A (CLASS DATA) 1 0x10D
0x000B (MAP LIST) 1 0x10E

<https://github.com/corkami/pics/blob/master/binary/Dex.png>

5

Binaries Android - .dex files

- Android - Dalvik executable format

<https://source.android.com/docs/core/runtime/dex-format>

- Rodrigo Chiossi. A deep dive into DEX file format

https://elinux.org/images/d/d9/A_deep_dive_into_dex_file_format--chiossi.pdf

- Ange Albertini. Dalvic EXecutable

<https://github.com/corkami/pics/blob/master/binary/DEX.png>

- Android - Dalvik executable instruction formats

<https://source.android.com/docs/core/runtime/instruction-formats>

- Android - Dalvik bytecode format

<https://source.android.com/docs/core/runtime/dalvik-bytecode>

- Android – ABIs

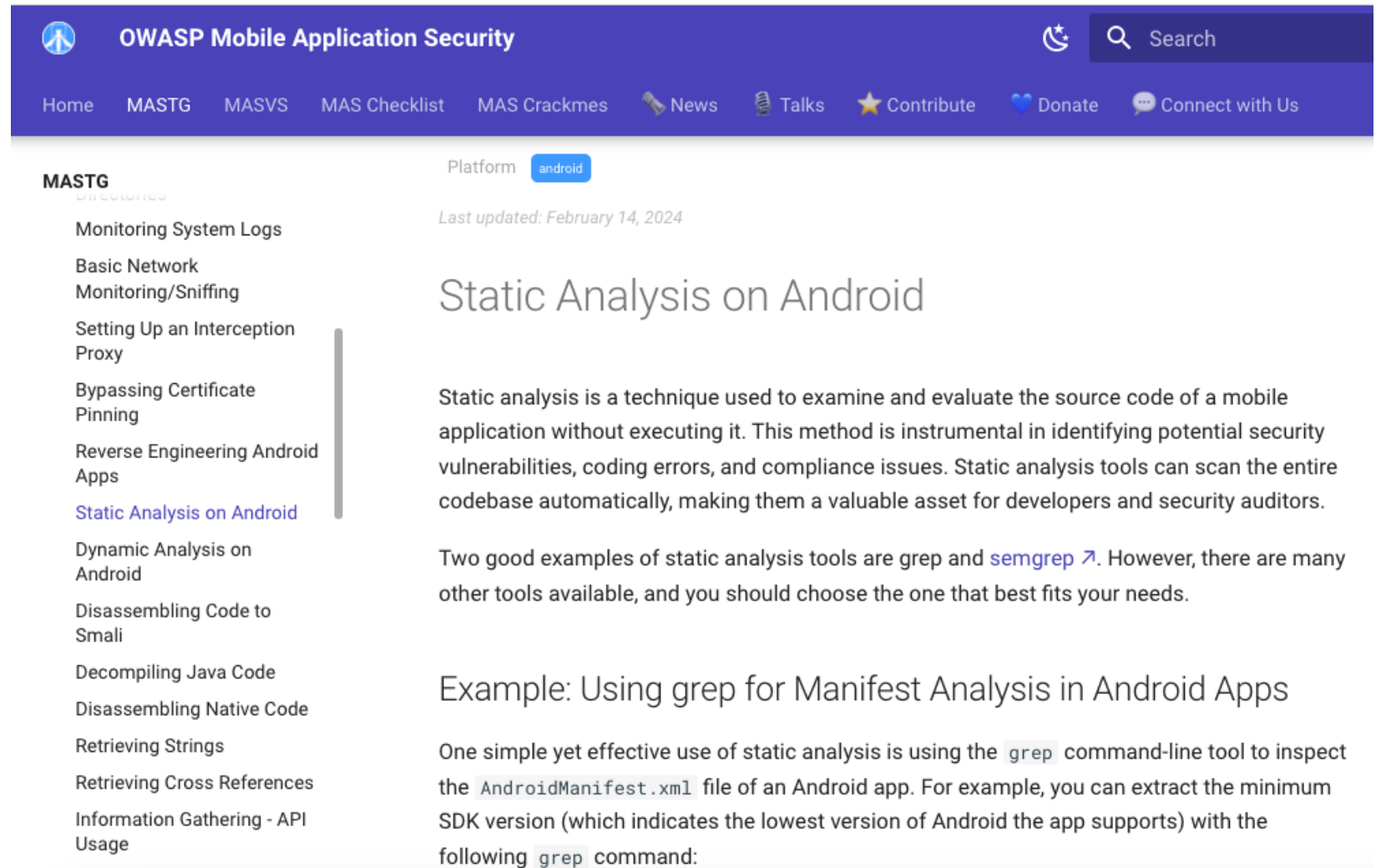
<https://developer.android.com/ndk/guides/abis>

CyberTruck'19 (21)

- <https://github.com/nowsecure/cybertruckchallenge19>
- <https://github.com/nowsecure/cybertruckchallenge19/tree/master/slides>



Static analysis



OWASP Mobile Application Security

Home MASTG MASVS MAS Checklist MAS Crackmes News Talks Contribute Donate Connect with Us

Platform **android**

Last updated: February 14, 2024

Static Analysis on Android

Static analysis is a technique used to examine and evaluate the source code of a mobile application without executing it. This method is instrumental in identifying potential security vulnerabilities, coding errors, and compliance issues. Static analysis tools can scan the entire codebase automatically, making them a valuable asset for developers and security auditors.

Two good examples of static analysis tools are `grep` and [semgrep](#). However, there are many other tools available, and you should choose the one that best fits your needs.

Example: Using `grep` for Manifest Analysis in Android Apps

One simple yet effective use of static analysis is using the `grep` command-line tool to inspect the `AndroidManifest.xml` file of an Android app. For example, you can extract the minimum SDK version (which indicates the lowest version of Android the app supports) with the following `grep` command:

<https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0014/>

Runtime RE

 **OWASP Mobile Application Security**  Search

Home MASTG MASVS MAS Checklist MAS Crackmes News Talks Contribute Donate Connect with Us

Platform **android**

↑ Back to top

Last updated: October 01, 2023

Runtime Reverse Engineering

Runtime reverse engineering can be seen as the on-the-fly version of reverse engineering where you don't have the binary data to your host computer. Instead, you'll analyze it straight from the memory of the app.

We'll keep using the HelloWorld JNI app, open a session with r2frida `r2`

```
frida://usb//sg.vantagepoint.helloworldjni
```

 and you can start by displaying the target binary information by using the `:i` command:

```
[0x00000000]> :i
arch      arm
bits      64
os         linux
pid        13215
uid        10096
objc       false
runtime    V8
```

<https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0045/>

Disassemble Decompile

The screenshot shows the OWASP Mobile Application Security website. The header is dark blue with the OWASP logo, the title "OWASP Mobile Application Security", a search bar, and navigation links: Home, MASTG, MASVS, MAS Checklist, MAS Crackmes, News, Talks, Contribute, Donate, and Connect with Us. The main content area has a left sidebar for the MASTG section, listing topics like Intro, Theory, Tests, Techniques (with sub-items: Generic, Android), Repackaging Apps, Installing Apps, Listing Installed Apps, Exploring the App Package, Accessing App Data Directories, Monitoring System Logs, and Basic Network. The main content area is titled "Decompiling Java Code" and includes a "Platform" filter set to "android" and a "Last updated: September 29, 2023" note. The text explains that in Android app security testing, if the application is based solely on Java and doesn't have any native code (C/C++ code), the reverse engineering process is relatively easy and recovers (decompiles) almost all the source code. It also mentions that black-box testing (with access to the compiled binary, but not the original source code) can get pretty close to white-box testing. Furthermore, it states that if the code has been purposefully obfuscated (or some tool-breaking anti-decompilation tricks have been applied), the reverse engineering process may be very time-consuming and unproductive. This also applies to applications that contain native code. They can still be reverse engineered, but the process is not automated and requires knowledge of low-level details. Finally, it suggests that if you want to look directly into Java source code on a GUI, you can simply open your APK using [jadx](#) or [Bytecode Viewer](#).

OWASP Mobile Application Security

Home MASTG MASVS MAS Checklist MAS Crackmes News Talks Contribute Donate Connect with Us

MASTG

- Intro
- Theory
- Tests
- Techniques
 - Generic
 - Android
 - Accessing the Device Shell
 - Host-Device Data Transfer
 - Obtaining and Extracting Apps
 - Repackaging Apps
 - Installing Apps
 - Listing Installed Apps
 - Exploring the App Package
 - Accessing App Data Directories
 - Monitoring System Logs
 - Basic Network

Platform **android**

Last updated: September 29, 2023

Decompiling Java Code

In Android app security testing, if the application is based solely on Java and doesn't have any native code (C/C++ code), the reverse engineering process is relatively easy and recovers (decompiles) almost all the source code. In those cases, black-box testing (with access to the compiled binary, but not the original source code) can get pretty close to white-box testing.

Nevertheless, if the code has been purposefully obfuscated (or some tool-breaking anti-decompilation tricks have been applied), the reverse engineering process may be very time-consuming and unproductive. This also applies to applications that contain native code. They can still be reverse engineered, but the process is not automated and requires knowledge of low-level details.

If you want to look directly into Java source code on a GUI, simply open your APK using [jadx](#) or [Bytecode Viewer](#).

<https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0017/>

iOS

- OWASP - Reverse Engineering iOS Apps

<https://mas.owasp.org/MASTG/techniques/ios/MASTG-TECH-0065/>

- Reverse Engineering iOS Applications

<https://github.com/ivRodriguezCA/RE-iOS-Apps>

Advanced

The screenshot shows the MASTG website interface. The top navigation bar is purple with a home icon, the title 'Mobile App Tampering and Reverse Engineering', a search bar, and links for Home, MASTG, MASVS, MAS Checklist, MAS Crackmes, News, Talks, Contribute, Donate, and Connect with Us. The left sidebar lists the MASTG content structure, with 'Mobile App Tampering and Reverse Engineering' highlighted in blue. The main content area is titled 'Advanced Techniques' and contains two paragraphs of text.

MASTG

- Intro >
- Theory >
 - General Concepts >
 - Mobile Application Taxonomy
 - Mobile Application Security Testing
 - Mobile App Tampering and Reverse Engineering**
 - Mobile App Authentication Architectures
 - Mobile App Network Communication
 - Mobile App Cryptography
 - Mobile App Code Quality
 - Mobile App User Privacy Protection
 - Android Security Testing >
 - iOS Security Testing >
 - Tests >

Advanced Techniques

For more complicated tasks, such as de-obfuscating heavily obfuscated binaries, you won't get far without automating certain parts of the analysis. For example, understanding and simplifying a complex control flow graph based on manual analysis in the disassembler would take you years (and most likely drive you mad long before you're done). Instead, you can augment your workflow with custom made tools. Fortunately, modern disassemblers come with scripting and extension APIs, and many useful extensions are available for popular disassemblers. There are also open source disassembling engines and binary analysis frameworks.

As always in hacking, the anything-goes rule applies: simply use whatever is most efficient. Every binary is different, and all reverse engineers have their own style. Often, the best way to achieve your goal is to combine approaches (such as emulator-based tracing and symbolic execution). To get started, pick a good disassembler and/or reverse engineering framework, then get comfortable with their particular features and extension APIs. Ultimately, the best way to get better is to get hands-on experience.

Dynamic Binary Instrumentation

Another useful approach for native binaries is dynamic binary instrumentations (DBI). Instrumentation frameworks such as Valgrind and PIN support fine-grained instruction-level

<https://mas.owasp.org/MASTG/General/0x04c-Tampering-and-Reverse-Engineering/#advanced-techniques>

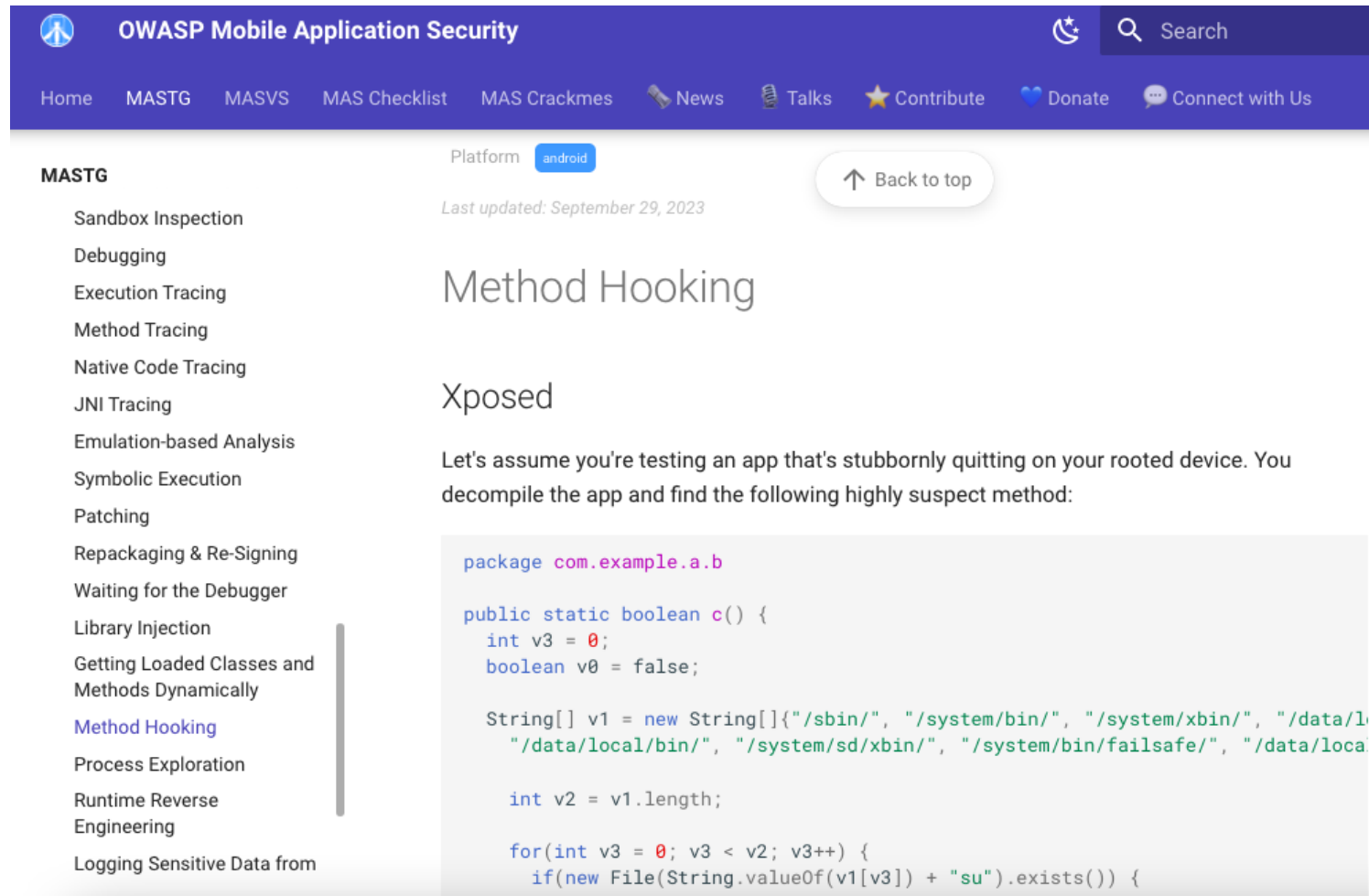
Hooking

” **Hooking** covers a wide range of **code modification methods** aimed at **altering the behavior of the mobile application** in question. This is done by **intercepting function calls, messages, or events passed between the software components**. **The code used for function interception** is called a **hook**. It applies to changing the behavior of operating systems and mentioned software components as well.”

<https://cybersecurity.asee.io/blog/mobile-application-hooking/>

- **Source modification** - altering the library source (through reverse engineering, before running the application)
- **Runtime modification** - inserting a hook while the application is running

Hooking



OWASP Mobile Application Security

Home MASTG MASVS MAS Checklist MAS Crackmes News Talks Contribute Donate Connect with Us

Platform **android**

Last updated: September 29, 2023

↑ Back to top

Method Hooking

Xposed

Let's assume you're testing an app that's stubbornly quitting on your rooted device. You decompile the app and find the following highly suspect method:

```
package com.example.a.b

public static boolean c() {
    int v3 = 0;
    boolean v0 = false;

    String[] v1 = new String[]{" /sbin/", "/system/bin/", "/system/xbin/", "/data/local/bin/", "/system/sd/xbin/", "/system/bin/failsafe/", "/data/local/"}

    int v2 = v1.length;

    for(int v3 = 0; v3 < v2; v3++) {
        if(new File(String.valueOf(v1[v3]) + "su").exists()) {
```

<https://mas.owasp.org/MASTG/techniques/android/MASTG-TECH-0043/>

<http://www.cydiasubstrate.com/api/c/MSHookFunction/>

Obfuscation

- OWASP – Testing obfuscation

<https://mas.owasp.org/MASTG/tests/android/MASVS-RESILIENCE/MASTG-TEST-0051/#dynamic-analysis>

- Android App reverse engineering 101

<https://www.ragingrock.com/AndroidAppRE/obfuscation.html>

Tools

Frida

FRIDA

[OVERVIEW](#)

[DOCS](#)

[NEWS](#)

[CODE](#)

[CONTACT](#)

Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers.

Scriptable

Inject your own scripts into black box processes. Hook any function, spy on crypto APIs or trace private application code, no source code needed. Edit, hit save,

Portable

Works on Windows, macOS, GNU/Linux, iOS, watchOS, tvOS, Android, FreeBSD, and QNX. Install the Node.js bindings from [npm](#), grab a Python package from [PyPI](#), or

Free

Frida is and will always be [free software](#) (free as in freedom). We want to empower the next generation of developer tools, and help other free software

Battle-tested

We are proud that [NowSecure](#) is using Frida to do fast, deep analysis of mobile apps [at scale](#). Frida has a comprehensive test-suite and has gone through

<https://frida.re/>

Dynamic analysis Valgrind

Information

About
News
Tool Suite
Supported Platforms
The Developers

Source Code

Current Releases
Release Archive
Variants / Patches
Code Repository
Valkyrie / GUIs

Documentation

Table of Contents
Quick Start
FAQ
User Manual
Download Manual
Research Papers
Books

Contact

Mailing Lists and IRC
Bug Reports
Feature Requests
Contact Summary
Commercial Support

How to Help

Contributing
Project Suggestions

Valgrind



Current release: [valgrind-3.22.0](#)

Valgrind is an instrumentation framework for building dynamic analysis tools. There are Valgrind tools that can automatically detect many memory management and threading bugs, and profile your programs in detail. You can also use Valgrind to build new tools.

The Valgrind distribution currently includes seven production-quality tools: a memory error detector, two thread error detectors, a cache and branch-prediction profiler, a call-graph generating cache and branch-prediction profiler, and two different heap profilers. It also includes an experimental SimPoint basic block vector generator. It runs on the following platforms: X86/Linux, AMD64/Linux, ARM/Linux, ARM64/Linux, PPC32/Linux, PPC64/Linux, PPC64LE/Linux, S390X/Linux, MIPS32/Linux, MIPS64/Linux, X86/Solaris, AMD64/Solaris, ARM/Android (2.3.x and later), ARM64/Android, X86/Android (4.0 and later), MIPS32/Android, X86/FreeBSD, AMD64/FreeBSD, ARM64/FreeBSD, X86/Darwin and AMD64/Darwin (Mac OS X 10.12).

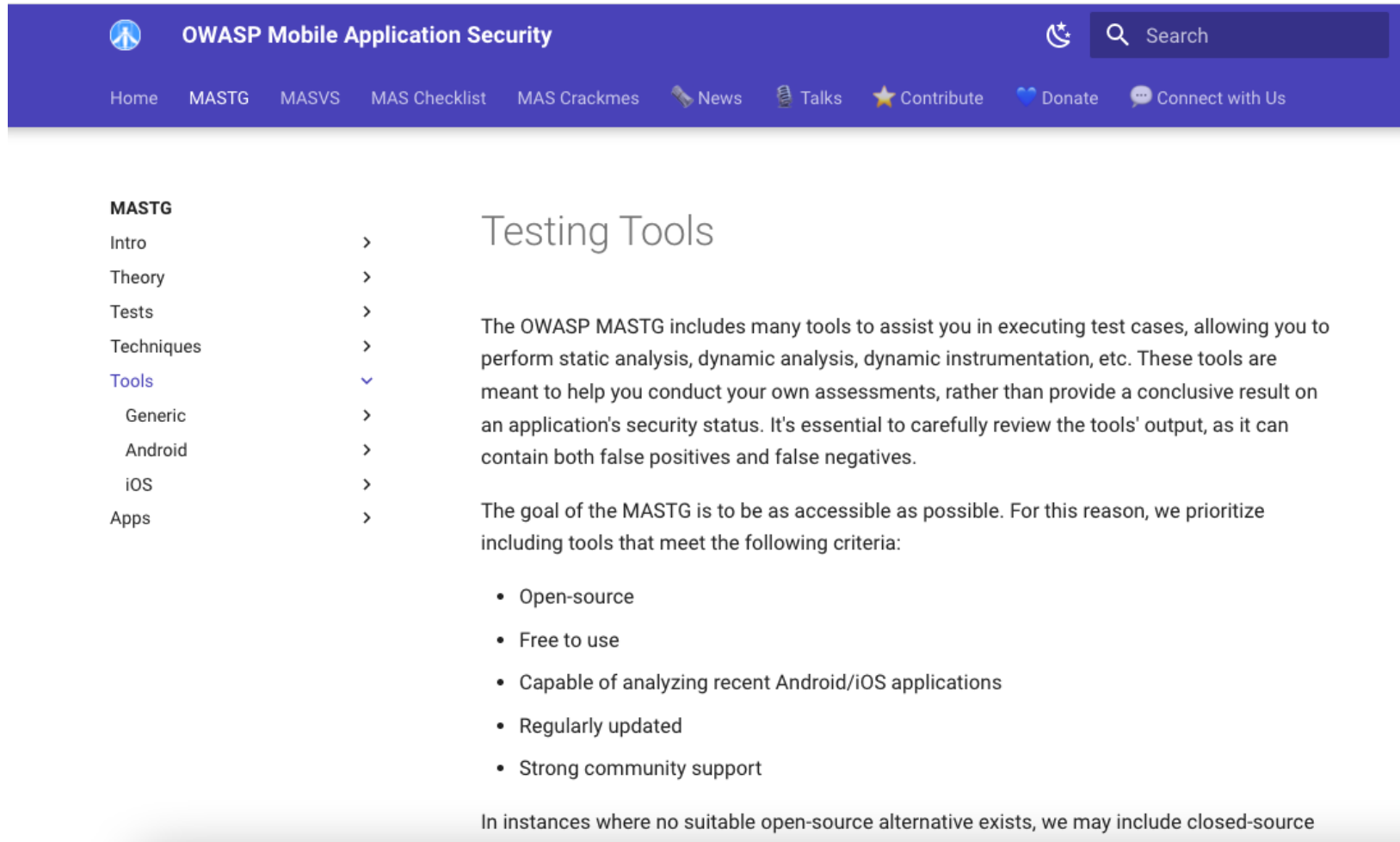
Valgrind is [Open Source](#) / [Free Software](#), and is freely available under the [GNU General Public License, version 2](#).

Recent News

<https://valgrind.org/>

<https://valgrind.org/docs/manual/dist.readme-android.html>

Tools



The screenshot shows the OWASP Mobile Application Security website. The header is dark blue with the OWASP logo and the title "OWASP Mobile Application Security". A search bar is on the right. The navigation menu includes Home, MASTG, MASVS, MAS Checklist, MAS Crackmes, News, Talks, Contribute, Donate, and Connect with Us. The left sidebar lists the MASTG sections: Intro, Theory, Tests, Techniques, Tools (highlighted with a blue arrow), Generic, Android, iOS, and Apps. The main content area is titled "Testing Tools" and contains two paragraphs of text and a bulleted list of criteria for tool selection.

MASTG

- Intro >
- Theory >
- Tests >
- Techniques >
- Tools** >
- Generic >
- Android >
- iOS >
- Apps >

Testing Tools

The OWASP MASTG includes many tools to assist you in executing test cases, allowing you to perform static analysis, dynamic analysis, dynamic instrumentation, etc. These tools are meant to help you conduct your own assessments, rather than provide a conclusive result on an application's security status. It's essential to carefully review the tools' output, as it can contain both false positives and false negatives.

The goal of the MASTG is to be as accessible as possible. For this reason, we prioritize including tools that meet the following criteria:

- Open-source
- Free to use
- Capable of analyzing recent Android/iOS applications
- Regularly updated
- Strong community support

In instances where no suitable open-source alternative exists, we may include closed-source

<https://mas.owasp.org/MASTG/tools/>

More resources

- OWASP Mobile Application Security - Mobile App Tampering and Reverse Engineering
<https://mas.owasp.org/MASTG/General/0x04c-Tampering-and-Reverse-Engineering/>
- R2Frida wiki mobile reverse engineering with r2frida
<https://github.com/nowsecure/r2frida/wiki>
(Presentations)
 - Mobile reverse engineering with r2frida – A beginner's workshop
[r2frida 4h training at r2con2020](#)
- user1342/Awesome-Android-Reverse-Engineering
<https://github.com/user1342/Awesome-Android-Reverse-Engineering?tab=readme-ov-file#Static-Analysis-Tools>
- Cybertruck challenge
<https://github.com/nowsecure/cybertruckchallenge19>
- Laurie wired
<https://www.youtube.com/@lauriewired>

More resources

- Fengwei Zhang, CSC 5991 Cyber Security Practice. Lab 5: Android Application Reverse Engineering and Obfuscation

<https://fengweiz.github.io/16sp-csc5991/labs/lab5-instruction.pdf>

- Defcon23 training

<https://github.com/rednaga/training/tree/master/DEFCON23>

- Android app Reverse Engineering 101

https://www.ragingrock.com/AndroidAppRE/app_fundamentals.html

- Introduction to ARM Assembly basics

<https://azeria-labs.com/writing-arm-assembly-part-1/>

- xth Working Conference on Reverse Engineering

Thanks!

- Good luck at the exam!